How to Scale a System from 0 to 10 million+ Users

ASHISH PRATAP SINGH · ·
JAN 29, 2026 2026 1 29

119 10 13 Shar

Scaling is a complex topic, but after working at **big tech** on services handling mi of requests and scaling my own **startup** (AlgoMaster.io) from scratch, I've realize most systems evolve through a surprisingly similar set of stages as they grow.

扩展系统是个复杂课题，但通过在大型科技公司处理数百万请求的服务经验，以 开始扩展自己的创业项目（AlgoMaster.io）后，我发现大多数系统在成长过程中 历惊人相似的扩展阶段。

The key insight is that **you should not over-engineer from the start**. Start simpl identify bottlenecks, and scale incrementally.

关键要诀是：不要从一开始就过度设计。保持简单起步，识别瓶颈，然后逐步扩

In this article, I'll walk you through **7 stages of scaling a system** from zero to 10 million users and beyond. Each stage addresses the specific bottlenecks that sho at different growth points. You'll learn what to add, when to add it, why it helps, the trade-offs involved.

本文将带你了解系统从零扩展到千万级用户的 7 个阶段。每个阶段都针对不同增 出现的特定瓶颈。你将掌握何时添加哪些组件、为何有效以及需要权衡的利弊。

Whether you're building an app or website, preparing for system design intervie just curious about how large-scale systems work, understanding this progression

sharpen they way you think about architecture.

无论你正在构建应用程序还是网站，准备系统设计面试，或只是好奇大型系统如何
作，理解这种演进过程将提升你的架构思维。

> **Disclaimer:** The user ranges and numbers mentioned in this article are approximate and intended to illustrate a scaling journey. Actual thresholds wi depending on your product, workload characteristics, and traffic patterns.
>
> 免责声明：本文提及的用户规模和数字均为近似值，旨在说明扩展历程。实际
> 因产品特性、工作负载特征和流量模式而异。

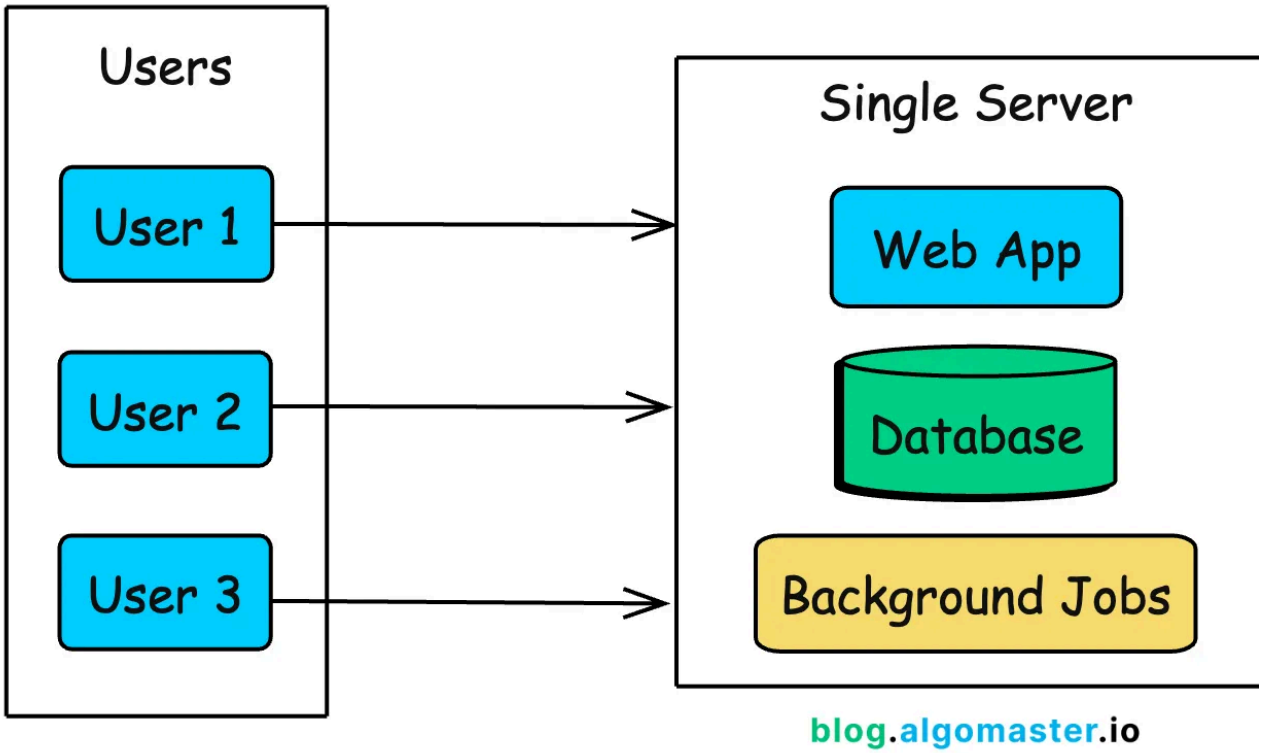
Stage 1: Single Server (0-100 User 0-100

When you're just starting out, your first priority is simple: **ship something and validate your idea**. Optimizing too early at this stage wastes time and money on problems you may never face.

当你刚开始时，首要任务很简单：推出产品并验证你的想法。在这个阶段过早优化
费时间和金钱在可能永远不会遇到的问题上。

The simplest architecture puts everything on a **single server**: your web applicatio database, and any background jobs all running on the same machine.

最简单的架构将所有内容放在单台服务器上：你的 Web 应用程序、数据库以及任
台任务都运行在同一台机器上。



> This is how Instagram started. When Kevin Systrom and Mike Krieger launch first version in 2010, 25,000 people signed up on day one.
>
> Instagram 就是这样起步的。2010 年凯文·斯特罗姆和迈克·克里格推出首个版本首日就获得了 2.5 万用户注册。
>
> They didn't over-engineer upfront. With a small team and a simple setup, they scaled in response to real demand, adding capacity as usage grew, rather than building for hypothetical future traffic.
>
> 他们没有过度设计前期架构。凭借小团队和简单配置，他们根据实际需求进行扩随着使用量增长逐步扩容，而非为假设的未来流量预先构建系统。

What This Architecture Looks Like

In practice, a single-server setup means:

实践中，单服务器配置意味着：

- A web framework (Django, Rails, Express, Spring Boot) handling HTTP requ
  处理 HTTP 请求的 Web 框架（Django、Rails、Express、Spring Boot）
- A database (PostgreSQL, MySQL) storing your data
  存储数据的数据库（PostgreSQL、MySQL）
- Background job processing (Sidekiq, Celery) for async tasks
  处理异步任务的后台作业系统（Sidekiq、Celery）
- Maybe a reverse proxy (Nginx) in front for SSL termination
  前端可能还需要反向代理（Nginx）来实现 SSL 终止

All of these run on one virtual machine. Your cloud provider bill might be $20-50/month for a basic VPS (DigitalOcean Droplet, AWS Lightsail, Linode).

所有这些都运行在一台虚拟机上。使用基础 VPS（DigitalOcean Droplet、AWS Lightsail、Linode）时，你的云服务账单可能为每月 20-50 美元。

Why This Works for Early Stage

At this stage, simplicity is your biggest advantage:

在这个阶段，简单性是你最大的优势：

- **Fast deployment**: One server means one place to deploy, monitor, and debug
  快速部署：单台服务器意味着只需关注一个部署、监控和调试点。
- **Low cost**: A single $20-50/month Virtual Private Server (VPS) can comfortab handle your first 100 users.
  低成本：每月 20-50 美元的虚拟专用服务器(VPS)就能轻松支撑前 100 名用户。
- **Faster iteration**: No distributed systems complexity to slow down developm
  迭代更快：没有分布式系统的复杂性拖慢开发进度。

- **Easier debugging**: All logs are in one place, and there are no network issues between components.

  调试更便捷：所有日志集中存储，组件间不存在网络问题。

- **Full-stack visibility**: You can trace every request end to end because there's one execution path.

  全栈可视化：由于仅存在单一执行路径，您可以端到端追踪每个请求。

The Trade-offs You're Making

This simplicity comes with trade-offs you accept knowingly:

这种简洁性伴随着您已知晓的妥协：

Trade-off	Impact	Why It's OK for Now
Single point of failure	Server down = app down	Few users, quick fixes acceptable
Resource contention	DB and app compete for CPU/RAM	Low traffic means no conten
No horizontal scaling	Can't add more servers easily	Traffic is predictable and low
Deployment downtime	Must restart for new code	Users won't notice brief outa

When to Move On

You'll know it's time to evolve when you notice these signs:

当出现以下迹象时，就意味着系统需要升级了：

- **Database queries slow down during peak traffic**: The app and database com for the same CPU and memory. One heavy query can drag down API latency

everyone.

高峰时段数据库查询变慢：应用程序和数据库会争夺相同的 CPU 和内存资源个复杂的查询就可能拖累所有人的 API 响应速度。

- **Server CPU or memory consistently exceeds 70-80%**: You're approaching th limits of what a single machine can reliably handle.

  服务器 CPU 或内存持续超过 70-80%：这表明单台服务器的处理能力已接近极

- **Deployments require restarts and cause downtime**: Even short interruption become noticeable, and users start to complain.

  部署需要重启并导致服务中断：即便是短暂的中断也会被用户察觉，并开始引诉。

- **A background job crash takes down the web server**: Without isolation, non-facing work can impact the user experience.

  后台任务崩溃导致网络服务器瘫痪：若缺乏隔离机制，非面向用户的工作可能用户体验。

- **You can't afford even brief downtime**: Your product has become critical enc that even maintenance windows stop being acceptable.

  你无法承受哪怕短暂的停机：产品已变得至关重要，甚至连维护窗口期都不可受。

At some point, the server starts to struggle under the weight of doing everything That's when it's time for your first architectural split.

当服务器开始因承担所有工作而不堪重负时，就是时候进行首次架构拆分了。

Stage 2: Separate Database (100-1 Users)

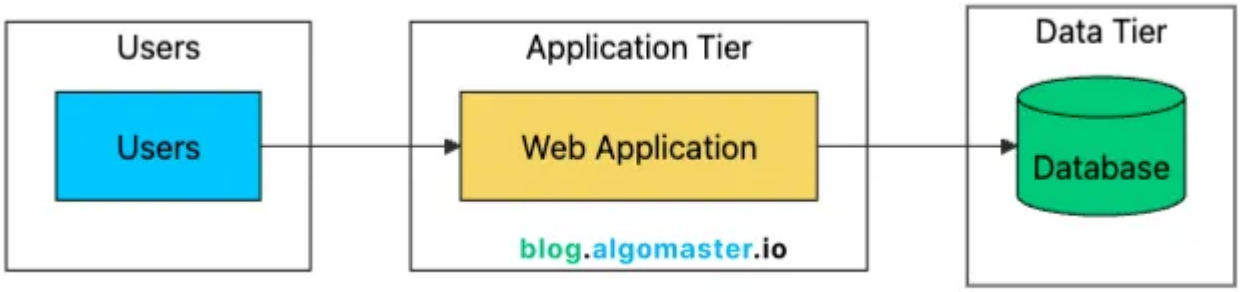
100-1K

As traffic grows, your single server starts struggling. The web application and da compete for the same CPU, memory, and disk I/O. A single heavy query can spik latency and slow down every API response.

随着流量增长，单台服务器开始不堪重负。Web 应用与数据库会争抢相同的 CPU 存和磁盘 I/O 资源。一条复杂查询就可能导致延迟激增，拖慢所有 API 响应速度。

The first scaling step is simple: **separate the database from the application serv**

第一步扩展方案很简单：将数据库与应用服务器分离。



This two-tier architecture gives you several immediate benefits:

这种双层架构能立即带来多项优势：

- **Resource Isolation:** Application and database no longer compete for CPU/memory. Each can use 100% of their allocated resources.

  资源隔离：应用与数据库不再争夺 CPU/内存资源，各自可 100%利用分配到的资源。

- **Independent Scaling:** Upgrade the database (more RAM, faster storage) witl touching the app server.

独立扩展：无需改动应用服务器即可升级数据库（增加内存、使用更快的存储
备）。

- **Better Security:** Database server can sit in a private network, not exposed to internet.

  安全性提升：数据库服务器可部署在私有网络中，避免直接暴露于互联网。

- **Specialized Optimization:** Tune each server for its specific workload. High for app server, high I/O for database.

  专项优化：根据工作负载特性调优服务器配置。应用服务器侧重高 CPU 性能
据库服务器侧重高 I/O 吞吐。

- **Backup Simplicity:** Database backups don't affect application performance s they run on a different machine.

  备份简易性：数据库备份不会影响应用程序性能，因为它们运行在不同的机器

Managed Database Services

At this stage, most teams use a managed database like **Amazon RDS**, **Google Clo SQL**, **Azure Database**, or **Supabase** (I use Supabase at algomaster.io).

在这个阶段，大多数团队会使用托管数据库服务，例如 Amazon RDS、Google Cl SQL、Azure Database 或 Supabase（我在 algomaster.io 使用的是 Supabase）。

Managed services typically handle:

托管服务通常提供以下功能：

- Automated backups (daily snapshots, point-in-time recovery)

  自动备份（每日快照、时间点恢复）

- Security patches and updates

  安全补丁与系统更新

- Basic monitoring and alerts

  基础监控与告警机制

- Optional read replicas (we'll cover these later)

  可选的只读副本（后续章节将详细说明）

- Failover to standby instances

  备用实例故障转移

The cost difference between self-hosting and managed is usually small once you in engineering time. A managed PostgreSQL instance might cost **$50–$100/mon more** than a raw VM, but it can save hours of maintenance every week. Those ho are better spent shipping features.

自建托管与使用托管服务的成本差异通常很小，若将工程师时间成本计算在内。扌 PostgreSQL 实例可能比裸机虚拟机每月贵 50-100 美元，但每周能节省数小时维扌——这些时间本可以用于开发新功能。

The main reasons to self-manage a database are:

选择自建数据库的主要原因包括：

- Cost optimization at very large scale

  超大规模下的成本优化

- Specific configurations that managed services don't support

  托管服务不支持的特定配置需求

- Compliance requirements that prohibit managed services

  禁止使用托管服务的合规要求

- You're building a database product

  您正在构建数据库产品

For most teams, managed services are the right choice until your database bill gr into the **thousands of dollars per month**.

对于大多数团队而言，在数据库月费用增长至数千美元之前，托管服务都是明智之

Connection Pooling

One often-overlooked improvement at this stage is connection pooling. Each dat connection consumes resources:

这一阶段常被忽视的优化点是连接池技术。每个数据库连接都会消耗资源：

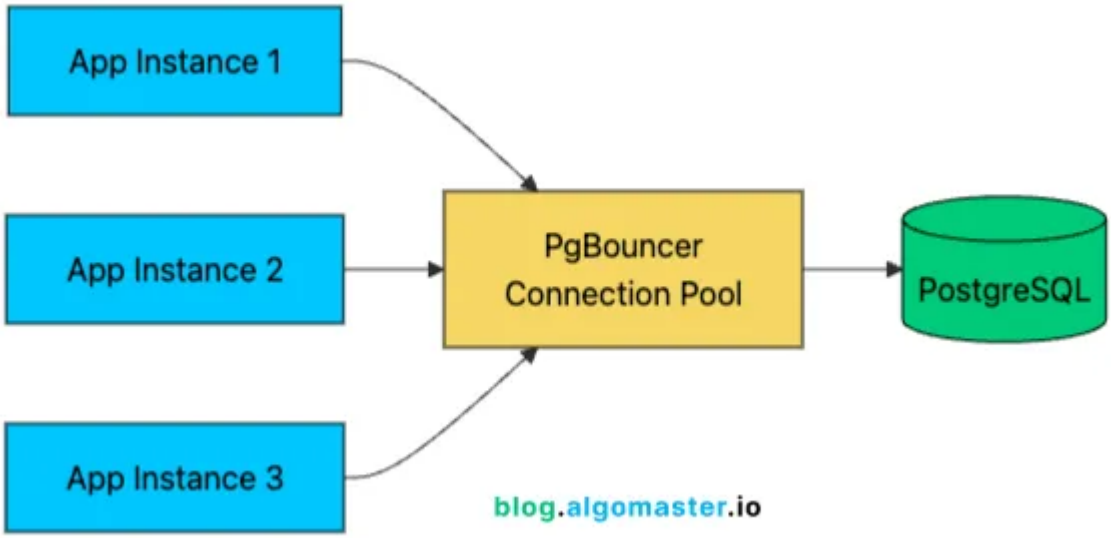
- Memory for the connection state (typically 5-10MB per connection in Postgr
  连接状态的内存占用（PostgreSQL 中通常每个连接需要 5-10MB）
- File descriptors on both app and database servers
  应用服务器和数据库服务器上的文件描述符
- CPU overhead for connection management
  连接管理的 CPU 开销

Opening a new connection is expensive too. Between the TCP handshake, SSL negotiation, and database authentication, you can add **50–100 ms** of overhead pe request.

建立新连接的成本也很高。TCP 三次握手、SSL 协商和数据库认证等步骤，会使 求增加 50-100 毫秒的开销。

A connection pooler like **PgBouncer** (for PostgreSQL) keeps a small set of databa connections open and reuses them across requests.

像 PgBouncer（适用于 PostgreSQL）这样的连接池工具会保持少量数据库连接处 启状态，并在多个请求间复用这些连接。



With 1,000 users, you might have 100 concurrent connections hitting your API. Without pooling, that's 100 database connections consuming resources. With po 20-30 actual database connections can efficiently serve those 100 application connections through connection reuse.

当系统拥有 1000 名用户时，API 可能面临 100 个并发连接请求。若不采用连接池 术，这意味着需要维持 100 个消耗资源的数据库连接。而通过连接池复用机制，仅 20-30 个实际数据库连接即可高效支撑这 100 个应用层连接请求。

Connection pooling modes:

连接池模式：

- **Session pooling**: One pool connection per client connection (most compatib least efficient)

  会话连接池：每个客户端连接对应一个池连接（兼容性最佳，效率最低）

- **Transaction pooling**: Connection returned to the pool after each transactior balance for most apps)

  事务连接池：每次事务完成后连接返回池中（最适合大多数应用的平衡方案）

- **Statement pooling**: Connection returned after each statement (most efficien can break features)

语句连接池：每条语句执行后连接立即返回池中（效率最高，但可能导致功能
常）

Most applications work best with **transaction pooling**, which often improves connection efficiency by **3–5x**.

大多数应用采用事务连接池效果最佳，通常能将连接效率提升 3-5 倍。

Network Latency Considerations

Separating the database introduces network latency. When app and database we
the same machine, “network” latency was essentially zero (loopback interface). N
every query adds 0.1-1ms of network round-trip time.

分离数据库会引入网络延迟。当应用和数据库在同一台机器时，"网络"延迟基本为
（回环接口）。现在每个查询都会增加 0.1-1 毫秒的网络往返时间。

For most applications, this is negligible. But if your code makes hundreds of data
queries per request (an anti-pattern, but common), this latency adds up. The solu
isn’t to put them back on the same machine, but to optimize your query patterns

对大多数应用而言，这种延迟可以忽略不计。但如果你的代码每个请求要执行数百
据库查询（虽然这是反模式，但很常见），这些延迟就会累积。解决方案不是将它
回同一台机器，而是优化查询模式：

- Batch queries where possible

  尽可能使用批量查询

- Use JOINs instead of N+1 query patterns

  使用 JOIN 操作替代 N+1 查询模式

- Cache frequently accessed data

  缓存频繁访问的数据

- Use connection pooling to avoid repeated connection setup overhead

  使用连接池来避免重复建立连接的开销

With the database on its own server, you've bought yourself room to grow. But yo also created a new single point of failure: the application server is now the weak What happens when it goes down, or when it simply can't keep up with demand?

将数据库部署在独立服务器上，你为自己赢得了扩展空间。但这也引入了一个新的 故障：应用服务器现在成了薄弱环节。当它宕机时，或者当它根本无法满足需求时 生什么？

Stage 3: Load Balancer + Horizonta Scaling (1K-10K Users)

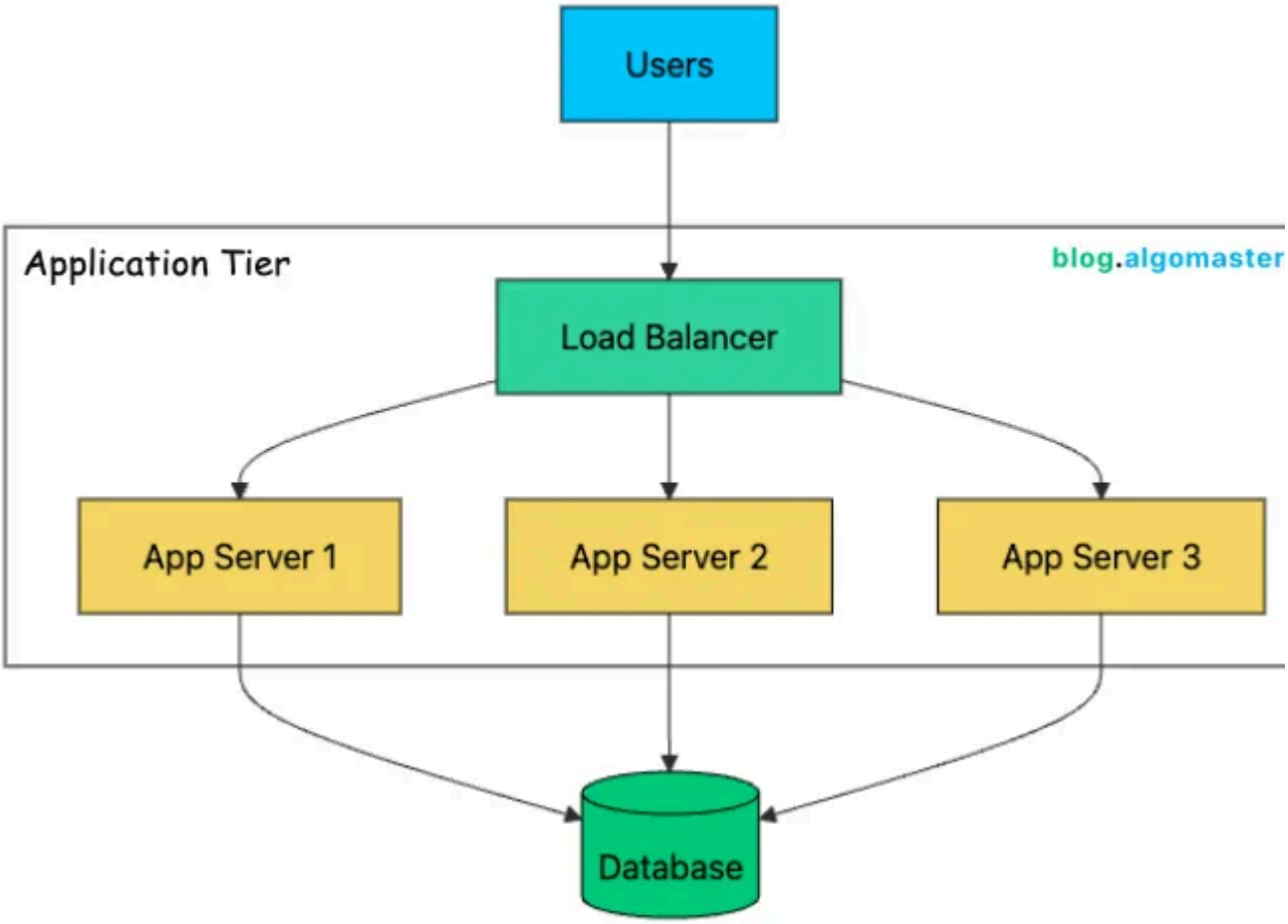
+ 1K-10K

Your separated architecture handles load better now, but you've introduced a new problem: your single application server is now a **single point of failure**. If it crash your entire application goes down. And as traffic grows, that one server can't kee

现在你的分离架构能更好地应对负载，但引入了一个新问题：单一应用服务器成了 故障。一旦它崩溃，整个系统就会瘫痪。随着流量增长，单台服务器将难以支撑。

The next step is to run **multiple application servers** behind a **load balancer**.

下一步是在负载均衡器后方部署多台应用服务器。



The load balancer sits in front of your servers and distributes incoming requests them. If one server fails, the load balancer detects this (via health checks) and rou traffic only to healthy servers. Users experience no downtime when a single serv fails.

负载均衡器位于服务器前端，将传入请求分配给各台服务器。当某台服务器发生故时，负载均衡器会通过健康检查发现异常，并将流量仅路由至正常运行的服务器。即使单台服务器宕机，用户也不会感受到服务中断。

The load balancer needs to decide which server handles each request. Common algorithms include: **Round Robin**, **Weighted Round Robin**, **Least Connections**, **Hash**, and **Random**.

负载均衡器需要决定由哪台服务器处理每个请求。常见的算法包括：轮询调度、加权轮询、最少连接、IP 哈希和随机分配。

Most teams start with Round Robin (simple, works well for most cases) and switc
Least Connections if they have requests with varying processing times.

大多数团队最初采用轮询调度（简单且适用于多数场景），若遇到处理时间差异轻
请求，则会切换到最少连接算法。

Modern load balancers operate at different layers:

现代负载均衡器在不同网络层级运行：

- **Layer 4** (**Transport**): Routes based on IP and port. Fast, but can't inspect HT
  headers.

  第四层（传输层）：基于 IP 和端口进行路由。速度快，但无法检查 HTTP 头
  息。

- **Layer 7** (**Application**): Routes based on HTTP headers, URLs, cookies. More
  flexible, slightly more overhead.

  第七层（应用层）：基于 HTTP 头部、URL 和 Cookie 进行路由。灵活性更高
  开销略大。

For most web applications, Layer 7 load balancing is preferable because it enable

对于大多数 Web 应用而言，第七层负载均衡更受青睐，因为它能够实现：

- Path-based routing (`/api/*` to API servers, `/static/*` to CDN)

  基于路径的路由（`/api/*` 指向 API 服务器，`/static/*` 指向 CDN）

- Header-based routing (different versions for mobile vs desktop)

  基于请求头的路由（移动端与桌面端不同版本）

- SSL termination at the load balancer

  在负载均衡器处终止 SSL

- Request/response inspection for security

  安全性的请求/响应检查

Vertical vs Horizontal Scaling

vs

Before adding more servers, you might ask: why not just get a bigger server? Thi
the classic vertical vs horizontal scaling trade-off.

在添加更多服务器之前，你可能会问：为什么不直接换台更大的服务器？这就是纟
垂直扩展与水平扩展之间的权衡。

Vertical scaling means moving to a larger server. It works well early on and usua
requires no code changes. But you eventually run into two problems: hard hardw
limits and rapidly increasing costs.

垂直扩展意味着升级到更强大的服务器。这种方式在早期效果良好，通常无需修改
码。但最终会遇到两个问题：硬件性能上限和成本急剧上升。

Bigger machines are priced non-linearly, so doubling CPU or memory can cost 3
more. And even the largest instances have a ceiling.

大型服务器的定价呈非线性增长，因此 CPU 或内存翻倍可能导致成本增加 3-4 倍
且即便是最大规格的实例也存在性能天花板。

Horizontal scaling means adding more servers. It is harder at first because your
application must be **stateless**, so any server can handle any request. But it gives y
effectively unlimited capacity and built-in redundancy. If one server fails, the sys
keeps running.

水平扩展意味着增加更多服务器。最初这会更困难，因为你的应用必须是无状态的
样任何服务器都能处理任何请求。但它能为你提供几乎无限的容量和内置冗余。如
台服务器出现故障，系统仍能继续运行。

The Session Problem

This is where horizontal scaling gets tricky. If a user logs in and their session liv
Server 1's **memory**, what happens when the next request lands on **Server 2**? Fron

app's perspective, the session is missing, so the user looks logged out.

这时水平扩展就变得棘手了。如果用户登录后，其会话保存在服务器 1 的内存中，当下一个请求落到服务器 2 上时会发生什么？从应用程序的角度看，会话丢失了，看起来就像退出了登录。

This is the **stateful server problem**, and it's the biggest obstacle to horizontal sca

这就是有状态服务器问题，也是水平扩展面临的最大障碍。

There are two common ways to handle it:

有两种常见的处理方式：

1. Sticky Sessions (Session Affinity)

1.

The load balancer routes all requests from the same user to the same server, typi using a cookie or IP hash.

负载均衡器将所有来自同一用户的请求路由到同一台服务器，通常使用 cookie 或希来实现。

Pros: 优点：

- Requires no application changes

  无需修改应用程序

- Works with any session storage

  兼容任何会话存储方式

Cons: 缺点：

- If that server fails, the user loses their session

  若该服务器故障，用户会话将丢失

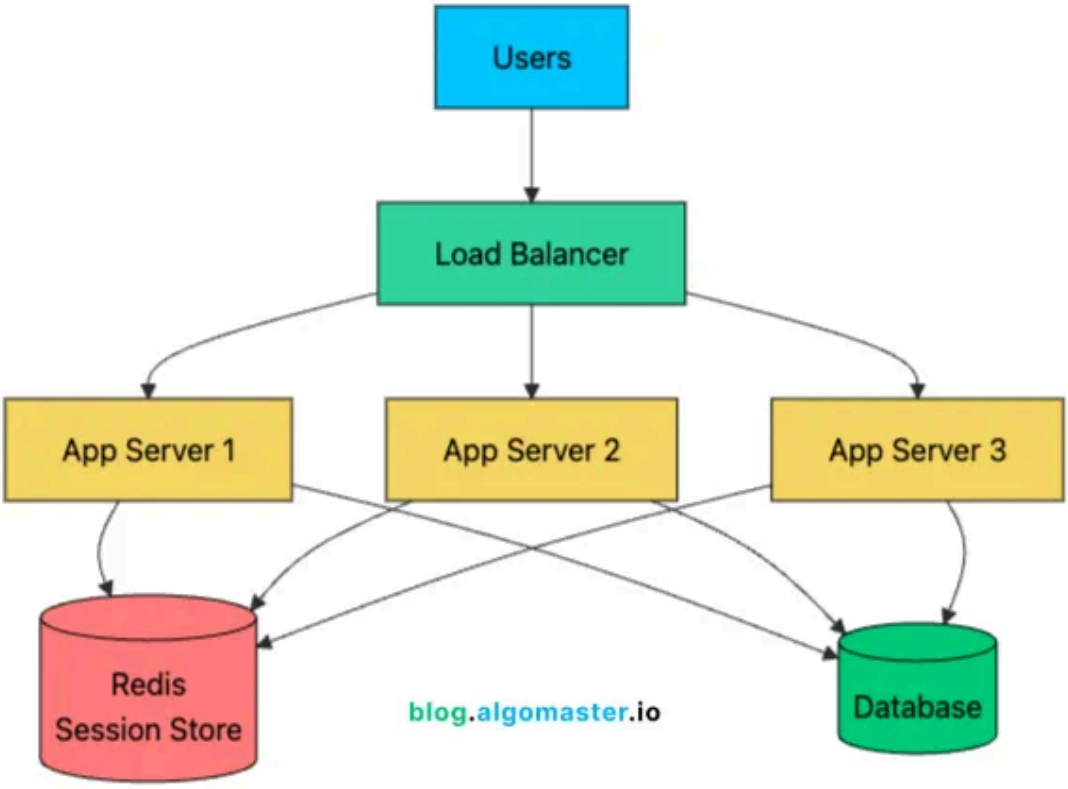
- Uneven load distribution if some users are more active than others
  用户活跃度不均导致的负载分配失衡
- Limits true horizontal scaling (can't freely move users between servers)
  限制了真正的水平扩展能力（无法在服务器间自由迁移用户）
- New servers take time to "warm up" with sessions
  新服务器需要时间通过会话"预热"

2. External Session Store

2.

Move session data out of the application servers into a shared store like **Redis** or **Memcached**.

将会话数据从应用服务器迁移到共享存储（如 Redis 或 Memcached）中



Now any server can handle any request because session data is centralized. This pattern most large-scale systems use. The added latency of a Redis lookup (sub-millisecond) is negligible compared to the flexibility it provides.

现在任何服务器都能处理任何请求，因为会话数据已集中存储。这是大多数大型系
用的架构模式。相比 Redis 查询（亚毫秒级）增加的开销，其带来的灵活性优势可
略不计。

You can now handle more traffic and survive server failures. But as your user bas grows, you'll notice something: no matter how many application servers you add, they're all hammering the same database. The database is becoming your next bottleneck.

现在你可以处理更多流量并应对服务器故障。但随着用户规模扩大，你会发现一个
象：无论添加多少应用服务器，它们都在访问同一个数据库。数据库正成为下一个
颈。

Stage 4: Caching + Read Replicas + CDN (10K-100K Users)

+ +CDN 1 -10

With 10,000+ users, a new bottleneck emerges: your database. Every request hits database, and as traffic grows, query latency increases. The database that handle QPS (queries per second) fine starts struggling at 1,000 QPS.

当用户突破 1 万时，新的瓶颈出现了：数据库。每个请求都会访问数据库，随着流
长，查询延迟不断增加。原本能轻松处理 100 QPS（每秒查询量）的数据库，在
QPS 时就开始不堪重负。

Read-heavy applications (which most are, with read-to-write ratios of 10:1 or hig suffer especially hard.

读密集型应用（大多数应用都属于此类，读写比例达到 10:1 甚至更高）受到的影 严重。

This stage introduces three complementary solutions: **caching**, **read replicas**, an **CDNs**. Together, they can reduce database load by 90% or more.

本阶段介绍三种互补解决方案：缓存、读取副本和 CDN。三者结合可将数据库负 低 90%以上。

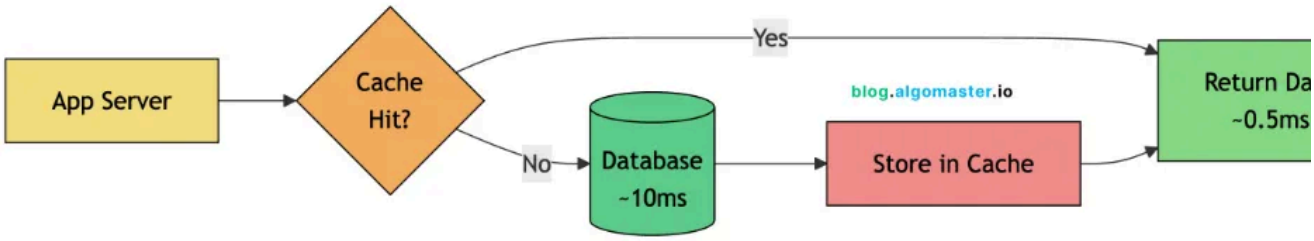
Caching Layer

Most web applications follow the 80/20 rule: 80% of requests access 20% of the da product page viewed 10,000 times doesn't need 10,000 database queries. The user profile that loads on every page view doesn't need to be fetched fresh each time.

大多数网络应用遵循 80/20 法则：80%的请求访问的是 20%的数据。一个被浏览 1 的产品页面不需要执行 1 万次数据库查询。每次页面加载时显示的用户资料也无 都重新获取。

Caching stores frequently accessed data in memory for near-instant retrieval. Wl database queries take 1-100ms, cache reads take 0.1-1ms.

缓存将频繁访问的数据存储在内存中，实现近乎即时的检索。数据库查询需要 1-1 秒，而缓存读取仅需 0.1-1 毫秒。



The most common caching pattern is **cache-aside** (also called lazy loading):

最常见的缓存模式是旁路缓存（也称为延迟加载）：

1. Application checks the cache first

   应用首先检查缓存

2. If data exists (cache hit), return it immediately

   如果数据存在（缓存命中），立即返回

3. If not (cache miss), query the database

   如果不存在（缓存未命中），则查询数据库

4. Store the result in cache for future requests (with TTL)

   将结果存入缓存供后续请求使用（设置 TTL）

5. Return the data 返回数据

Redis and Memcached are the standard choices here. Redis is more feature-rich (supports data structures like lists, sets, sorted sets; persistence; pub/sub; Lua scripting), while Memcached is simpler and slightly faster for pure key-value cac

Redis 和 Memcached 是此处的标准选择。Redis 功能更丰富（支持列表、集合、
合等数据结构；持久化；发布/订阅；Lua 脚本），而 Memcached 更简单，在纯
存场景下速度略快。

Most teams choose Redis because the additional features are useful (using sorted for leaderboards, lists for queues, etc.), and the performance difference is negligi

大多数团队选择 Redis，因为其附加功能非常实用（使用有序集合实现排行榜、列
现队列等），且性能差异可以忽略不计。

What to Cache

Not everything should be cached. Good cache candidates include:

并非所有内容都适合缓存。适合缓存的候选数据包括：

Data Type	TTL	Reason
User profiles	5-15 minutes	Loaded on every page, changes infrequently
Product listings	1-5 minutes	High read volume, tolerable staleness
API responses	1-60 seconds	Reduce expensive computations
Database query results	Varies	Avoid repeated expensive queries
Session data	30 minutes	Required for every authenticated request
Rate limiting counters	1 minute	High volume, accuracy matters less

Poor cache candidates: 不适合缓存的候选数据：

- Highly personalized data (different for every user, low reuse)
  高度个性化的数据（每个用户不同，复用率低）
- Frequently changing data (constant invalidation overhead)
  频繁变动的数据（持续失效带来的开销）
- Large blobs (consumes memory without proportional benefit)
  大型数据块（占用内存但收益不成正比）
- Transactional data where staleness causes issues
  会导致数据过期问题的交易数据

Cache Invalidation

The hardest part of caching isn't adding it, it's keeping it accurate. When underly data changes, cached data becomes stale. This is famously one of the "two hard problems in computer science."

缓存最困难的部分不在于添加它，而在于保持其准确性。当底层数据发生变化时，数据就会过时。这被公认为"计算机科学中两大难题"之一。

Common strategies include:

常见策略包括：

Strategy	How It Works	Trade-offs
TTL (Time-to-Live)	Data expires after N seconds automatically	Simple, but stale data possib within TTL window
Write-Through	Update cache immediately when database updates	Always fresh, but write latenc increases
Write-Behind	Update cache, async write to DB later	Fast writes, but data loss risk cache fails
Cache-Aside with Invalidation	Delete cache key when data changes	Explicit control, but cache mi on next read
Event-Based	Listen to database changes, invalidate accordingly	Precise, but requires event infrastructure (CDC)

Most systems start with TTL-based expiration (set cache to expire after 5-60 min and add explicit invalidation for data where staleness causes problems. For exam

大多数系统最初采用基于 TTL 的过期机制（将缓存设置为 5-60 分钟后失效），然对那些数据陈旧会导致问题的场景添加显式失效机制。例如：

```
def update_user_profile(user_id, new_data):
    # Update database
    db.update("users", user_id, new_data)
    # Invalidate cache
    cache.delete(f"user:{user_id}")
```

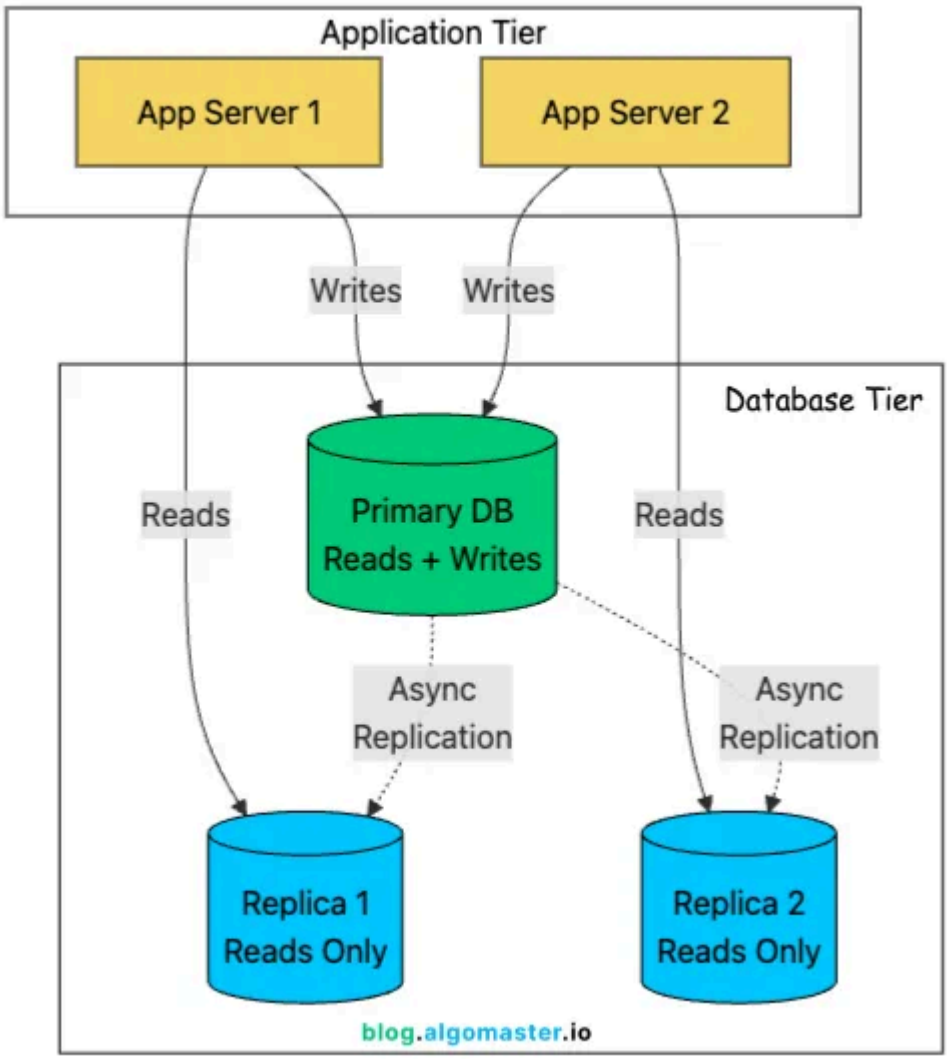
The next read will miss the cache and fetch fresh data from the database.

下一次读取将无法命中缓存，转而从数据库获取最新数据。

Read Replicas

Even with caching, some requests will still hit the database, especially **writes** anc **cache misses**. Read replicas help by distributing read traffic across multiple copi the database.

即便使用了缓存，部分请求仍会访问数据库，特别是写入操作和缓存未命中的情况取副本通过将读取流量分散到数据库的多个副本来缓解这一问题。



The primary database handles all writes. Changes are then replicated (usually asynchronously) to one or more **read replicas**. Your application sends read queri replicas and keeps the write workload on the primary, which reduces contention improves overall throughput.

主数据库负责处理所有写入操作。随后这些变更会被（通常以异步方式）复制到 多个只读副本。应用程序将读查询发送至副本，而将写工作负载保留在主数据库 而减少争用并提高整体吞吐量。

Replication Lag

One important consideration is **replication lag**. Since replication is often asynchronous (for performance), replicas might be milliseconds to seconds behir

primary.

一个重要的考量因素是复制延迟。由于复制通常是异步的（为了性能），副本可能
主数据库延迟几毫秒到几秒。

For most applications, this is acceptable. If a social media feed is a second behin
most users will not notice. But some flows require stronger consistency.

对大多数应用而言，这种延迟是可接受的。如果社交媒体动态晚一秒更新，大多数
不会察觉。但某些流程需要更强的一致性保障。

A common failure mode is **read-your-writes consistency**:

常见的故障模式是读写一致性：

A user updates their profile and refreshes immediately. If that read lands on a re
that has not caught up, they see old data and assume the update failed.

用户更新个人资料后立即刷新页面。若该读取请求落在尚未同步的副本节点上，他
看到旧数据，误以为更新失败。

Solutions: 解决方案：

1. **Read from primary after writes**: For a short window (N seconds) after a writ
route that user's reads to the primary.

   写后主库读取：写入后的短暂时间窗口（N 秒内），将该用户的读取请求路由
   库。

2. **Session-level consistency**: Track the user's last write timestamp and only rea
from replicas that have caught up past that point.

   会话级一致性：追踪用户最后写入的时间戳，仅从已同步超过该时间点的副本
   数据。

3. **Explicit read-from-primary**: For critical reads (viewing just-updated data), a
hit the primary.

   显式主库读取：对于关键读取（查看刚更新的数据），始终访问主库。

Most frameworks have built-in support for read/write splitting. For example, Rai (ActiveRecord), Django, and Hibernate can route reads to replicas and writes to t primary automatically.

大多数框架内置了读写分离支持。例如 Rails（ActiveRecord）、Django 和 Hibe 都能自动将读请求路由到副本，写请求路由到主库。

Content Delivery Network (CDN)

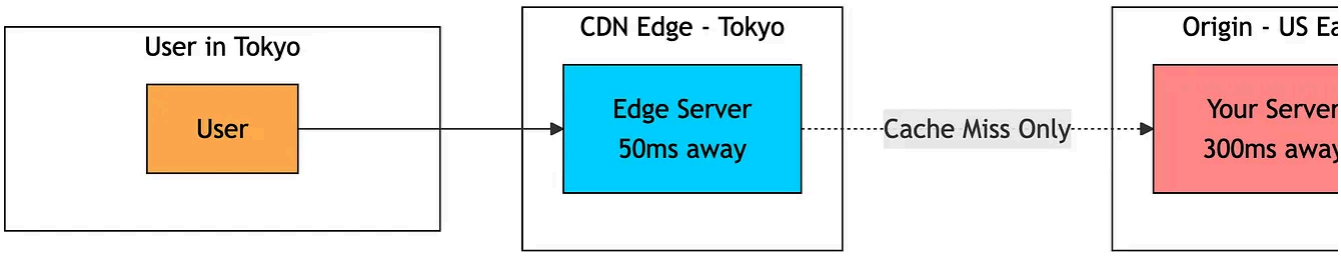
CDN

Static assets like images, CSS, JavaScript, and videos rarely change and don't nee hit your application servers at all. They're also the largest files you serve, which r them expensive in both bandwidth and compute if you serve them directly.

图片、CSS、JavaScript 和视频等静态资源很少变动，完全无需经过应用服务器处 这些文件通常体积最大，若直接由服务器提供，将消耗大量带宽和计算资源。

A **CDN** solves this by caching static assets on globally distributed servers called **locations** (or points of presence).

内容分发网络（CDN）通过在全球分布的边缘节点（又称接入点）缓存静态资源 这一问题。



Here's what happens when a user in Tokyo requests an image:

以下是东京用户请求图片时的处理流程：

- The request is routed to the **CDN edge in Tokyo** (low latency, say ~50 ms rou trip).

请求被路由至东京的 CDN 边缘节点（低延迟，往返约 50 毫秒）。

- If the file is already cached (**cache hit**), the CDN serves it immediately.

  若文件已缓存（缓存命中），CDN 会立即返回该文件。

- If it's not cached (**cache miss**), the CDN fetches it from your **origin** (maybe i US, ~300 ms), stores a copy at the edge, and then returns it to the user.

  若未缓存（缓存未命中），CDN 会从源站（可能位于美国，约 300 毫秒延迟 取文件，在边缘节点存储副本后返回给用户。

- The next user in Tokyo gets the cached version from the edge, again at ~50 n

  东京的下一位用户从边缘节点获取缓存版本，响应时间同样约为 50 毫秒。

Popular CDNs include **Cloudflare** (strong free tier), **AWS CloudFront**, **Fastly**, an **Akamai**.

热门 CDN 服务包括 Cloudflare（提供优质免费套餐）、AWS CloudFront、Fastl Akamai。

With caching, read replicas, and a CDN in place, your system can handle steady growth. The next challenge is **spiky traffic**. A viral post, a marketing campaign, even the difference between 3 AM and 3 PM can create 10x traffic variation. At t point, manually adjusting capacity stops working.

通过引入缓存、读取副本和 CDN，您的系统已能应对稳定增长。接下来要应对的 量突增问题——病毒式传播的帖子、营销活动，甚至是凌晨 3 点与下午 3 点的差 可能引发 10 倍的流量波动。此时手动调整容量已不再可行。

Stage 5: Auto-Scaling + Stateless Design (100K-500K Users)

10 + -50

At 100K+ users, traffic patterns become less predictable. You might have:

当用户突破 10 万大关时，流量模式变得难以预测。您可能会遇到：

- Daily peaks (morning in US, evening in EU)
  日常高峰时段（美国早晨/欧洲晚间）
- Weekly patterns (higher on weekdays for B2B, weekends for consumer)
  工作日流量较高（B2B 业务），周末流量较高（消费者业务）
- Marketing campaign spikes (10x traffic for hours)
  营销活动流量激增（短时间内出现 10 倍流量）
- Viral moments (100x traffic, unpredictable duration)
  病毒式传播时刻（100 倍流量，持续时间不可预测）

At this point, manually adding and removing servers is no longer viable. You nee infrastructure that reacts automatically.

此时，手动增减服务器已不可行。你需要能自动响应的基础设施。

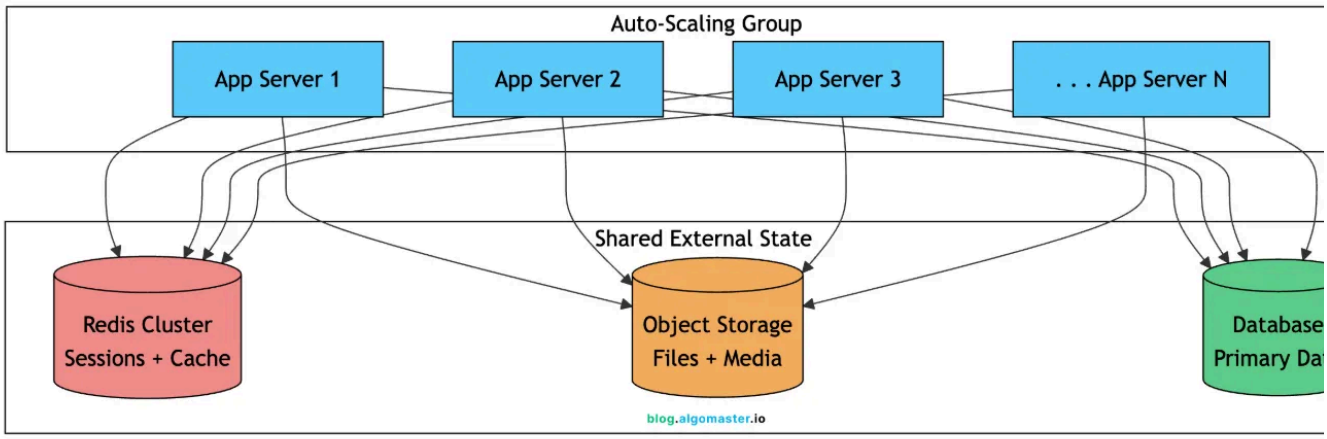
This stage focuses on **auto-scaling** (automatically adjusting capacity) and ensurir your application is truly **stateless** (servers can be added or removed freely withou loss or user impact).

这一阶段重点关注自动扩展（自动调整容量）和确保应用实现真正的无状态化（可增减服务器而不会造成数据丢失或影响用户体验）。

Stateless Architecture

For auto-scaling to work, your application servers must be interchangeable. Any request can go to any server. Any server can be terminated without losing data. A server can start handling requests immediately.

要实现自动扩展，应用服务器必须具有可互换性。任何请求都能被任意服务器处理
何服务器终止时都不会丢失数据。新服务器能立即开始处理请求。



When a new server joins the cluster, it typically:

当新服务器加入集群时，通常会发生以下情况：

1. Starts the application 启动应用程序
2. Registers with the load balancer (or gets discovered)
   向负载均衡器注册（或被自动发现）
3. Connects to Redis, database, and other shared services
   连接 Redis、数据库及其他共享服务
4. Immediately starts handling requests
   立即开始处理请求

When a server is removed:

当服务器被移除时：

1. Load balancer stops sending new requests

   负载均衡器停止发送新请求

2. In-flight requests complete (graceful shutdown)

   正在处理的请求完成（优雅关闭）

3. Server terminates 服务器终止

No data is lost, because nothing important is stored locally.

数据不会丢失，因为重要信息都不存储在本地。

Auto-Scaling Strategies

Auto-scaling adjusts capacity based on metrics. The scaling system continuously monitors metrics and adds or removes servers based on thresholds.

自动扩缩容根据指标动态调整容量。该扩缩系统持续监控各项指标，并基于阈值自减服务器数量。

Most teams start with CPU-based scaling. It's simple, works for most workloads, is easy to reason about. Add queue-depth scaling for background job workers.

多数团队最初会采用基于 CPU 的扩缩策略。这种方式简单直观，适用于大多数工载，且易于理解。对于后台任务处理程序，可额外增加队列深度作为扩缩依据。

Scaling Parameters

When configuring auto-scaling, you'll set these parameters:

配置自动扩缩时，需要设置以下参数：

```
Minimum instances: 2       # Always running, even at zero traffic
Maximum instances: 20      # Cost ceiling and resource limit
Scale-up threshold: 70%    # CPU percentage to trigger scale-up
Scale-down threshold: 30%  # CPU percentage to trigger scale-down
Scale-up cooldown: 3 min   # Wait time after scaling up before next action
```

```
Scale-down cooldown: 10 min # Wait time after scaling down
Instance warmup: 2 min     # Time for new instance to become fully operational
```

Important considerations:

重要注意事项：

- **Minimum instances**: Should be at least 2 for redundancy. If one fails, the oth handles traffic while a replacement spins up.

  最小实例数：至少应保持 2 个以实现冗余。若其中一个实例故障，另一个可处 量，同时启动新实例进行替换。

- **Cooldown periods**: Prevent thrashing (rapidly scaling up and down). Scale-d cooldown is typically longer because removing capacity is riskier than addin

  冷却周期：防止系统抖动（频繁扩容缩容）。缩容冷却期通常更长，因为减少 比增加容量的风险更高。

- **Instance warmup**: New servers need time to start, load code, warm up cache establish database connections. Don't count them toward capacity until they ready.

  实例预热：新服务器需要时间启动、加载代码、预热缓存、建立数据库连接。 备就绪前不应将其计入容量。

- **Asymmetric scaling**: Scale up aggressively (react quickly to load), scale down conservatively (don't remove capacity too soon).

  非对称扩缩：激进扩容（快速响应负载），保守缩容（避免过早减少容量）。

JWT for Stateless Authentication

JWT

At this scale, many teams move from session-based to token-based authenticatio using JWTs (JSON Web Tokens). With session-based auth, every request requires

session store lookup. With JWTs, authentication state is contained in the token i

在这一规模下，许多团队会从基于会话的认证转向使用 JWT（JSON Web Tokens 基于令牌的认证。基于会话的认证需要每次请求都查询会话存储，而 JWT 则将认 态直接包含在令牌本身中。

A JWT has three parts:

JWT 由三部分组成：

```
Header.Payload.Signature

eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxMjM0NTZ9.signature_here
```

The payload contains claims like user ID, roles, and expiration. The signature en the token wasn't tampered with. Any server can verify the signature using a shar secret key without querying a database.

载荷部分包含用户 ID、角色和过期时间等声明信息。签名部分确保令牌未被篡改 何服务器都可通过共享密钥验证签名，无需查询数据库。

Trade-offs with JWTs: 使用 JWT 的权衡考量：

- **Pro**: Truly stateless, no session store lookup for every request

  优势：真正无状态，无需为每个请求查询会话存储

- **Pro**: Works across services (microservices, mobile apps, third-party APIs)

  优势：跨服务适用（微服务、移动应用、第三方 API）

- **Con**: Can't invalidate individual tokens before expiry (user logs out, but toke remains valid)

  劣势：无法在到期前使单个令牌失效（用户已登出，但令牌仍有效）

- **Con**: Token size adds to each request (500 bytes vs 32-byte session ID)

  劣势：令牌体积增加每个请求的负载（500 字节 vs 32 字节的会话 ID）

A common pattern is **short-lived access tokens** (for example, 15 minutes) plus **lo lived refresh tokens** (for example, 7 days). That limits how long a compromised c token can be used.

一种常见模式是采用短期访问令牌（例如 15 分钟）配合长期刷新令牌（例如 7 天 这种机制能限制被盗或失效令牌的使用时长。

At this point, your application tier scales elastically. Traffic spikes and more serv spin up. Traffic drops and they spin down.

此时，您的应用层已实现弹性扩展。流量激增时自动启动更多服务器，流量回落时 动缩减。

But a new ceiling is coming: the database can only handle so many writes, the monolith becomes harder to change safely, and some operations are too slow to r synchronously. That's when you bring in the heavy machinery.

但新的瓶颈即将出现：数据库写入能力存在上限，单体架构难以安全修改，部分同 作耗时过长。这时就需要引入重型解决方案了。

Stage 6: Sharding + Microservices + Message Queues (500K-1M Users) + 50 -100

With 500K+ users, you'll hit new ceilings that the previous optimizations can't sc

当用户量突破 50 万时，之前的优化方案将遇到新的瓶颈：

- Writes overwhelm a single primary database, even if reads are offloaded to replicas.

即便读操作已分流至副本库，主数据库仍会因写入压力过载

- The monolith becomes painful to ship. A small change to notifications force full redeploy of the entire application.

  单体架构的部署变得极其痛苦。仅需修改通知功能的小改动，就不得不重新部个应用系统

- Previously fast operations start taking seconds because too much work is happening synchronously in the request path.

  原本快速的操作开始需要数秒才能完成，因为请求路径中有太多工作以同步方行。

- Different parts of the product need different scaling profiles. Search and fee may need 10x the capacity of profile pages.

  产品的不同部分需要不同的扩展方案。搜索和动态信息流可能需要十倍于个人页面的容量。

This is where the heavy machinery comes in: **database sharding**, **microservices**, **asynchronous processing** (message queues).

这时就需要动用重型技术方案：数据库分片、微服务和异步处理（消息队列）。

Database Sharding

Read replicas solved read scaling, but writes all still go to one primary database. high volume, this primary becomes the bottleneck. You're limited by what one machine can handle in terms of:

只读副本解决了读取扩展问题，但所有写入操作仍指向主数据库。在高负载情况个主库会成为瓶颈。单台机器的处理能力限制了以下方面：

- Write throughput (inserts, updates, deletes)

  写入吞吐量（插入、更新、删除操作）

- Storage capacity (even big disks have limits)

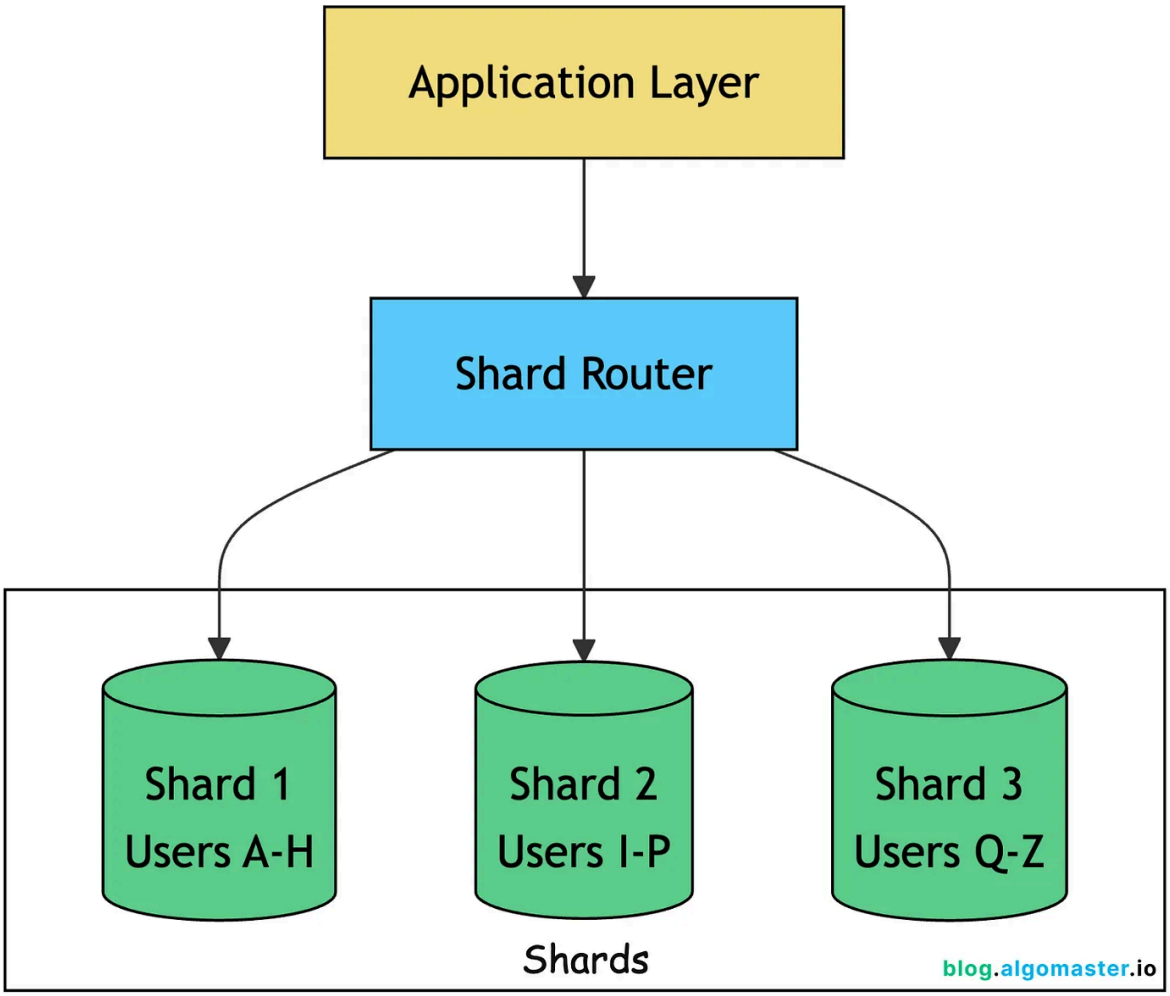
  存储容量（再大的磁盘也有上限）

- Connection count (even with pooling)

  连接数（即使使用连接池）

Sharding splits your data across multiple databases based on a **shard key**. Each s holds a subset of the data and handles both reads and writes for that subset.

分片技术根据分片键将数据分散到多个数据库中。每个分片存储数据的一个子集，责处理该子集的读写操作。



Sharding Strategies

Strategy	How It Works	Pros	Cons
Hash-based	`hash(user_id) % num_shards`	Even distribution across shards	Adding shards requ data migration (all c rehashed)
Range-based	Users 1-1M in shard 1, 1M-2M in shard 2	Simple implementation, range queries work within shard	Uneven load if som ranges are more ac
Directory-based	Lookup table maps each key to a shard	Maximum flexibility, can move individual keys	Lookup table becor bottleneck, adds latency
Geographic	Users in US on shard 1, EU on shard 2	Low latency, data residency compliance	Uneven distribution complex if users mo regions

> **Consistent hashing** is a popular improvement over simple hash-based shardi
> Instead of `hash(key) % num_shards`, you place keys on a ring. When you new shard, only keys adjacent to its position move, not all keys. This means ac a fourth shard moves ~25% of data instead of ~75%.
>
> 一致性哈希是对简单基于哈希的分片方法的一种流行改进。不同于传统方式，
> 分布在一个环形空间上。当新增分片时，只有与其位置相邻的键需要迁移，而
> 数据。这意味着添加第四个分片时仅需移动约 25%的数据，而非约 75%。

When to Shard

Sharding is a **one-way door**. Once you shard:

分片是一条单行道。一旦实施分片：

- Cross-shard queries become expensive or impossible (joining data across sh

  跨分片查询会变得昂贵或无法实现（需要跨分片连接数据）

- Transactions spanning shards are complex (two-phase commit or give up on atomicity)

  跨分片事务处理变得复杂（必须使用两阶段提交或放弃原子性）

- Schema changes must be applied to all shards

  所有分片都必须同步执行模式变更

- Operations (backups, migrations) multiply by shard count

  操作（备份、迁移）随分片数量成倍增加

- Application code becomes more complex (shard routing logic)

  应用程序代码变得更复杂（需处理分片路由逻辑）

Before sharding, exhaust these options:

实施分片前，请先尝试以下优化方案：

1. **Optimize queries**: Add missing indexes, rewrite slow queries, denormalize w helpful

   优化查询：补充缺失的索引、重写低效查询、在必要时采用反规范化设计

2. **Vertical scaling**: Upgrade to a bigger database server (more CPU, RAM, fast SSDs)

   垂直扩展：升级到更强大的数据库服务器（增加 CPU、内存、更快的固态硬

3. **Read replicas**: If read-heavy, add replicas to handle reads

   读取副本：若读取压力大，可添加副本来分担查询负载

4. **Caching**: Reduce load on database by caching frequently accessed data

   缓存机制：通过缓存高频访问数据来减轻数据库压力

5. **Archival**: Move old data to cold storage (separate database, object storage)

   数据归档：将旧数据迁移至冷存储（独立数据库或对象存储）

6. **Connection pooling**: Reduce connection overhead

   连接池：降低连接开销

Only shard when you're truly write-bound and a single node physically cannot ha
your throughput, or when your dataset exceeds what fits on one machine.

仅当真正遇到写入瓶颈且单个节点物理上无法处理吞吐量时，或当数据集超过单机
时才进行分片

Microservices

As the product and team grow, a monolith becomes harder to evolve safely. Com
signals you might benefit from microservices:

随着产品和团队规模扩大，单体架构会越来越难以安全演进。以下迹象表明微服务
带来益处：

- A change to one area (like notifications) requires redeploying the entire app.

  修改单一功能（如通知系统）需要重新部署整个应用

- Teams can't ship independently without coordinating every release.

  各团队必须协调每次发布才能独立交付功能

- Different parts of the app have different scaling needs (search needs 10 serve
  profile viewing needs 2)

  应用不同模块的扩展需求各异（搜索需要 10 台服务器，个人主页浏览仅需 2

- Engineers frequently conflict in the same codebase.

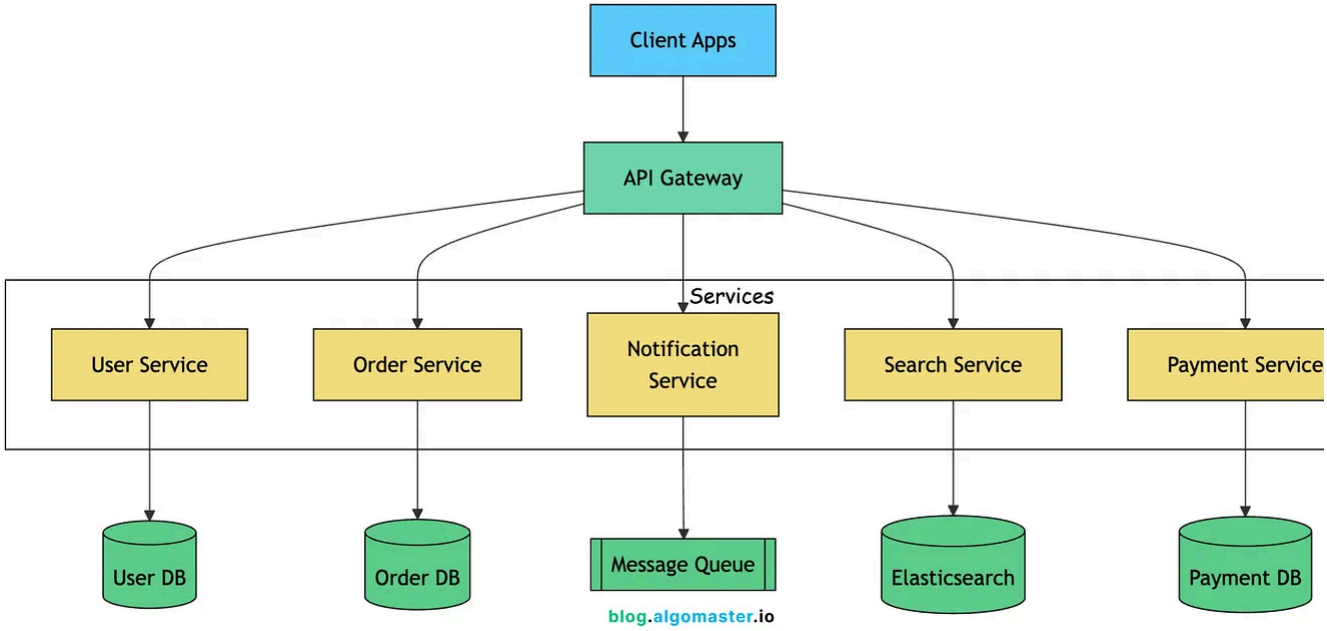
  工程师经常在同一个代码库中产生冲突。

- A bug in one subsystem takes down the whole application.

  某个子系统的故障会导致整个应用程序崩溃。

Microservices split the application into independent services that communicate
the network.

微服务架构将应用程序拆分为通过网络通信的独立服务。



Each service: 每个服务：

- **Owns its data** (a database only it writes to directly)

  拥有自己的数据（仅由它直接写入的数据库）

- **Deploys independently** (ship notifications without touching checkout)

  独立部署（无需修改结账系统即可推送通知）

- **Scales independently** (search can scale separately from profiles)

  独立扩展（搜索服务可与用户档案服务分开扩容）

- **Uses fit-for-purpose tech** (search might use Elasticsearch, payments might r Postgres with strong consistency)

  采用专用技术（搜索可能使用 Elasticsearch，支付系统可能需要强一致性的 Postgres 数据库）

- **Exposes a clear API contract** (other services integrate via stable endpoints)

  提供明确的 API 契约（其他服务通过稳定端点进行集成）

The trade-off is a big jump in operational complexity. The safest approach is to s with **one extraction**: pick the service with the cleanest boundaries and the cleare

independent scaling needs. Avoid splitting into dozens of services upfront.

代价是运维复杂度的大幅提升。最稳妥的做法是从单一服务拆分入手：选择边界最
晰、独立扩展需求最明确的服务进行解耦。切忌一开始就拆分成数十个微服务。

Message Queues and Async Processing

Not everything needs to happen synchronously in the request path. When a user
an order, some steps must complete immediately, while others can happen in the
background.

并非所有操作都需要在请求路径中同步完成。当用户下单时，部分步骤必须实时处
而其他操作可以放在后台异步执行。

Must be synchronous: 必须同步处理的环节：

- Validate payment method 验证支付方式
- Check inventory 检查库存
- Create order record 创建订单记录
- Return order confirmation
  返回订单确认信息

Can be asynchronous: 可以是异步的：

- Send confirmation email 发送确认邮件
- Update analytics dashboard
  更新分析仪表盘
- Notify warehouse for fulfillment
  通知仓库进行订单处理

- Update recommendation engine

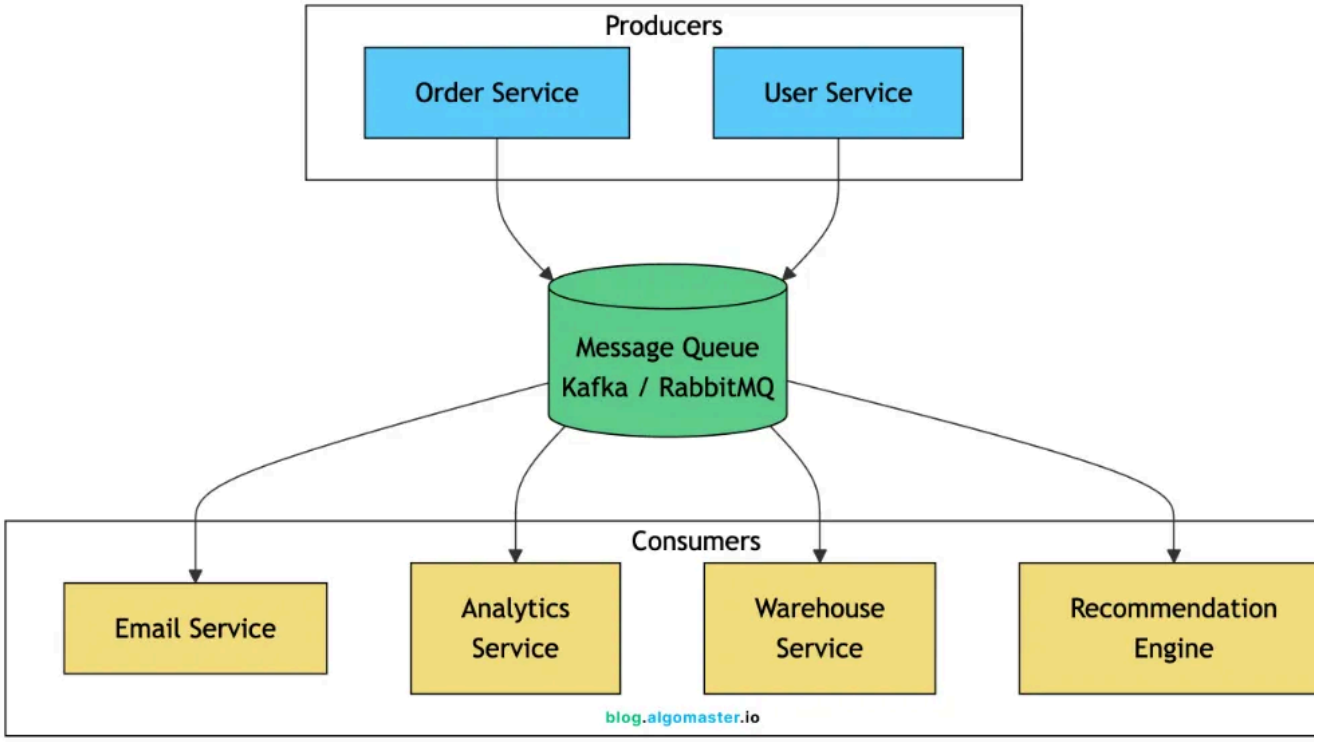
  更新推荐引擎

- Sync to accounting system

  同步至财务系统

Message queues like **Kafka**, **RabbitMQ**, or **SQS** decouple producers from consun
The order service publishes an event like `OrderPlaced`, and downstream syster
consume it independently.

诸如 Kafka、RabbitMQ 或 SQS 等消息队列实现了生产者与消费者的解耦。订单
布类似 `OrderPlaced` 的事件后，下游系统可独立消费该事件。



Benefits of async processing:

异步处理的优势：

- **Resilience**: If email service is down, messages queue up. Order still complete
  Email sends when service recovers.

弹性设计：当邮件服务中断时，消息会自动排队。订单仍能顺利完成，待服务
后补发邮件。

- **Scalability**: Consumers scale independently based on queue depth. Holiday r
Add more warehouse notification processors without touching the orders se

  可扩展性：消费者能根据队列深度独立扩展。遇到节日高峰？只需增加仓库通
理器，无需修改订单服务。

- **Decoupling**: The order service doesn't need to know who consumes the ever
can add a new consumer (fraud detection, CRM sync) without changing the
producer.

  解耦设计：订单服务无需知晓事件消费者。您可以新增消费者（如欺诈检测、
同步）而无需改动生产者代码。

- **Smoothing bursts**: Queues absorb spikes and let downstream systems proce
sustainable rate instead of getting overloaded.

  缓冲峰值：队列能吸收流量洪峰，让下游系统以可持续的速率处理请求，避免
载。

- **Retry handling**: Failed messages can be retried automatically. Dead letter qu
capture messages that fail repeatedly for investigation.

  重试处理：失败的消息可以自动重试。死信队列会捕获多次失败的消息以供调

A common real-world pattern is "do the write now, do the heavy work later."

现实世界中一个常见模式是"先执行写入操作，繁重工作稍后处理"。

For example, in social apps, creating a post is usually a fast write and an immedi
success response. Expensive work like fan-out, indexing, notifications, and feed
updates happens asynchronously, which is why you sometimes see small delays i

counts or feed propagation.

例如在社交应用中，发布内容通常能快速写入并立即获得成功响应。而诸如粉丝扩
内容索引、通知发送和动态更新等耗时操作会异步执行，这就是为什么有时你会看
赞数或动态传播出现短暂延迟。

At this point, your architecture can handle massive scale within a single region. B
your users aren't all in one place, and neither should your infrastructure be.

此时，您的架构已能在单一区域内应对海量规模。但用户并非集中于一处，您的基
施也不应如此。

Once you have users across continents, latency becomes noticeable, and a single
datacenter becomes a single point of failure for your entire global user base.

当用户遍布各大洲时，延迟问题就会显现，单一数据中心将成为全球用户群的单点
风险。

Stage 7: Multi-Region + Advanced Patterns (1M-10M+ Users)

100 1000 +

With millions of users worldwide, new challenges emerge:

当用户规模扩展至全球数百万级别时，新的挑战随之而来：

- Users in Australia experience 300ms latency hitting US servers

  澳大利亚用户访问美国服务器时延高达 300 毫秒

- A datacenter outage (fire, network partition, cloud provider issue) takes dow
  entire service

单数据中心故障（火灾、网络分区、云服务商问题）将导致全服务中断

- Your database schema can't efficiently serve both write-heavy real-time upd and read-heavy analytics dashboards

  你的数据库架构无法同时高效处理写入密集的实时更新和读取密集的分析仪表

- Different regions have different data residency requirements (GDPR in EU, localization laws)

  不同地区有不同的数据驻留要求（欧盟的 GDPR 法规、数据本地化法律）

This stage covers **multi-region deployment**, **advanced caching**, and **specialized patterns** like CQRS.

这一阶段涉及多区域部署、高级缓存以及 CQRS 等专业模式。

Multi-Region Architecture

Deploying to multiple geographic regions achieves two main goals:

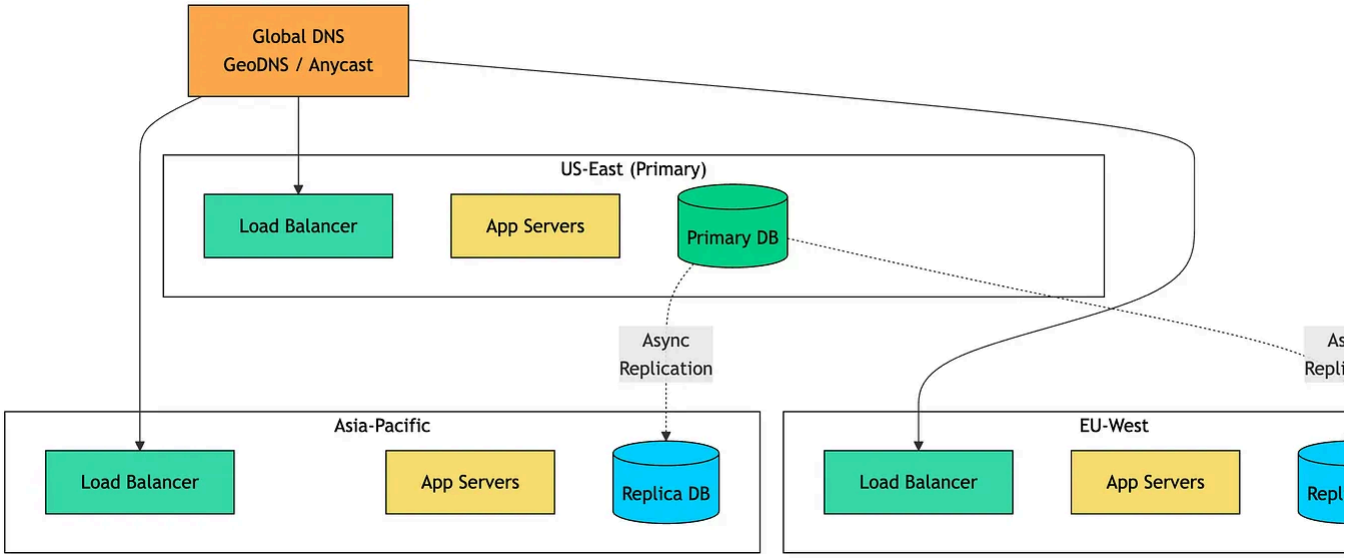
部署到多个地理区域主要实现两个目标：

1. **Lower latency**: Users connect to nearby servers. Tokyo users hit Tokyo serve (20ms) instead of US servers (200ms).

   降低延迟：用户连接最近的服务器。东京用户访问东京服务器（20 毫秒）而 服务器（200 毫秒）。

2. **Disaster recovery**: If one region fails, others continue serving traffic. True h availability.

   灾难恢复：若某区域发生故障，其他区域仍可继续处理流量。真正实现高可



There are two main approaches:

主要有两种方法：

Active-Passive (Primary-Secondary)

-

One region (primary) handles all writes. Other regions serve reads and can take c
the primary fails.

单个区域（主节点）处理所有写入操作。其他区域负责读取，并可在主节点故障时
服务。

Pros: 优势：

- Simpler to implement 实现更简单
- No write conflict resolution needed

  无需解决写入冲突
- Strong consistency for writes

  写入强一致性

Cons: 缺点：

- Higher write latency for users far from primary

  远离主节点的用户写入延迟较高

- Failover isn't instantaneous (DNS propagation, replica promotion)

  故障转移并非即时生效（DNS 传播、副本升级需要时间）

- Primary region is still a single point of failure

  主区域仍是单点故障

Active-Active

All regions handle both reads and writes. This requires solving the hard problem
happens when users in US and EU update the same record simultaneously?

所有区域都同时处理读取和写入操作。这需要解决一个棘手的问题：当美国和欧洲
户同时更新同一条记录时会发生什么？

Pros: 优点：

- Lowest possible latency for all operations

  所有操作的最低延迟

- True high availability, any region failure is seamless

  真正的高可用性，任何区域故障都能无缝切换

- No single point of failure

  无单点故障

Cons: 缺点：

- Conflict resolution is complex (and can cause data issues if done wrong)

  冲突解决机制复杂（若处理不当可能导致数据问题）

- Eventually consistent, not suitable for all data types

  最终一致性，并非适用于所有数据类型

- More complex to reason about and debug

  系统逻辑更复杂，调试难度更高

Most companies start with active-passive. Active-active requires solving distribu consensus problems and accepting eventual consistency.

多数企业最初采用主备架构。主主架构需要解决分布式共识问题并接受最终一致性

CAP Theorem at Global Scale

CAP

The CAP theorem becomes very real at global scale. It states that a distributed sy can only provide two of three guarantees:

在全球化部署中，CAP 定理变得尤为现实。该定理指出分布式系统只能同时满足三项中的两项：

- **Consistency**: Every read receives the most recent write

  一致性：每次读取都能获取最新写入的数据

- **Availability**: Every request receives a response (not an error)

  可用性：每个请求都能获得响应（非错误响应）

- **Partition Tolerance**: System continues despite network partitions

  分区容错性：系统在网络分区时仍能持续运行

Since network partitions between regions are inevitable (undersea cables get cut providers have outages), you're really choosing between consistency and availabi during a partition.

由于区域间的网络分区不可避免（海底电缆被切断、云服务商出现故障），实际_ 在分区期间选择一致性或可用性。

Most global systems choose **eventual consistency** for most operations:

大多数全球性系统对多数操作选择最终一致性：

- A user's post might take 1-2 seconds to appear for followers in other regions

  用户的帖子可能需要 1-2 秒才会显示给其他区域的关注者

- A product rating might show slightly different averages in different regions briefly

  产品评分在不同地区可能短暂显示略微不同的平均值

- User profile updates might take a moment to propagate

  用户资料更新可能需要片刻才能同步完成

Only operations where inconsistency causes real problems (payments, inventory decrements, financial transactions) require strong consistency, and those might r to a primary region.

只有那些不一致会导致实际问题（支付、库存扣减、金融交易）的操作才需要强一性，这些操作可能会被路由到主区域执行。

CQRS Pattern CQRS

As systems grow, read and write patterns diverge significantly:

随着系统规模扩大，读写模式会出现显著分化：

- Writes need transactions, validation, normalized data, audit logs

  写入操作需要事务支持、数据验证、规范化数据结构和审计日志

- Reads need denormalized data, fast aggregations, full-text search

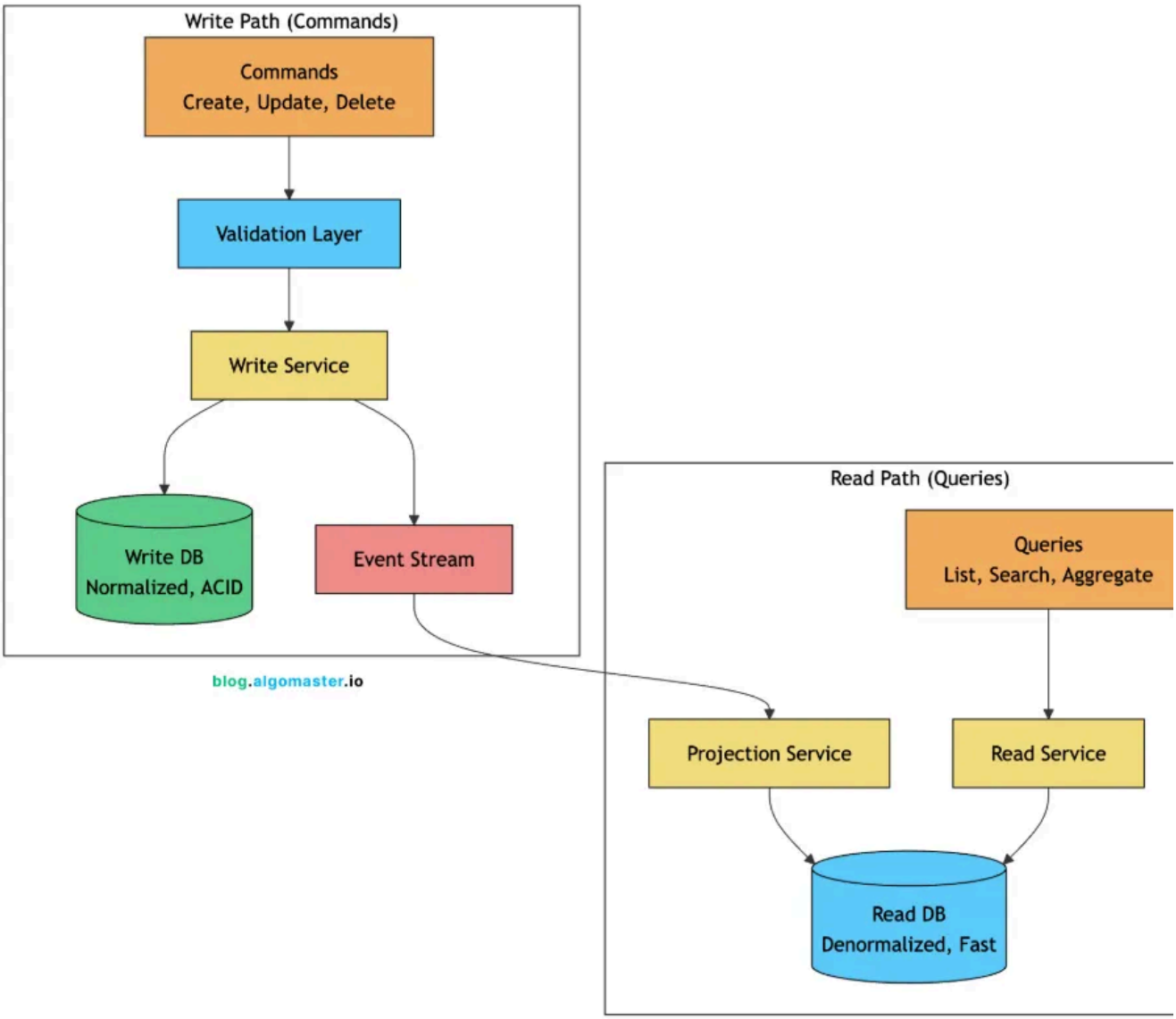
  读取操作需要非规范化数据、快速聚合和全文搜索功能

- Write volume might be 1/100th of read volume

  写入量可能仅为读取量的百分之一

CQRS (Command Query Responsibility Segregation) separates these concerns entirely.

CQRS（命令查询职责分离）模式彻底分离了这两类操作



The write side uses a normalized schema optimized for data integrity and transactional guarantees. The read side uses denormalized views optimized for q performance. Events synchronize the two.

写入端采用规范化数据模型，确保数据完整性和事务性；读取端使用非规范化视图 查询性能。事件机制保持两者同步。

Real-world example: Twitter's timeline architecture.

现实案例：Twitter 时间线架构。

- **Write path**: When you tweet, it's written to a normalized tweets table with p indexing, constraints, and transactions.

  写入路径：当你发布推文时，它会被写入一个规范化的推文表，该表具有适当 引、约束和事务处理。

- **Event**: A "tweet created" event fires.

  事件：触发"推文创建"事件。

- **Projection**: A fan-out service reads the event and adds the tweet to each foll timeline (a denormalized, per-user data structure optimized for "show me m feed" queries).

  投影：扇出服务读取该事件，并将推文添加到每个关注者的时间线中（这是一 规范化的、按用户优化的数据结构，专为"显示我的动态"查询而设计）。

- **Read path**: When you open Twitter, you read from your pre-computed timel not a complex query joining tweets, follows, and users.

  读取路径：当你打开 Twitter 时，你读取的是预先计算好的时间线，而不是通 杂查询连接推文、关注关系和用户数据。

CQRS adds complexity but enables:

CQRS 增加了复杂性，但能实现：

- Independent scaling of read and write paths

  读写路径的独立扩展

- Optimized schemas for each access pattern

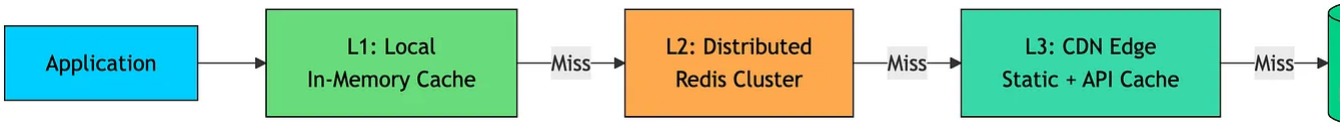
  为每种访问模式优化数据结构

- Different technology choices (PostgreSQL for writes, Elasticsearch for reads
  差异化技术选型（写操作用 PostgreSQL，读操作用 Elasticsearch）
- Better performance for both operations
  提升两项操作的性能

Advanced Caching Patterns

At global scale, caching becomes more sophisticated:
在全球规模下，缓存策略变得更加复杂：

Multi-Tier Caching



Layer	Type	Latency	Size	Use Case
L1	Application memory	< 0.1ms	Small (GBs)	Hot data, request-scop
L2	Distributed (Redis)	0.5-2ms	Medium (10s-100s GB)	Shared data, sessions
L3	CDN edge	10-50ms	Large (TBs)	Static content, API responses
Origin	Database	10-100ms	Very large	Authoritative data

Cache Warming

When a new cache server starts (or cache expires after maintenance), the first req
face cache misses, causing latency spikes and origin load. Cache warming pre-

populates caches before traffic arrives:

当新缓存服务器启动时（或维护后缓存过期），首批请求会遭遇缓存未命中，导致
激增和源站负载。缓存预热技术能在流量到达前预先填充缓存：

- **On deployment**: Load popular items into cache during startup, before receiv traffic

  部署阶段：在接收流量前，启动时就将热门内容加载至缓存

- **Before campaigns**: Before a marketing push, warm caches with products/pa likely to be accessed

  营销活动前：在推广攻势前，预先加热可能被访问的商品/页面缓存

- **Cache replication**: When adding a new cache node, copy state from existing

  缓存复制：在添加新缓存节点时，从现有节点复制状态

> Netflix pre-warms edge caches with popular content before peak hours. Wher evening viewing starts, the most-watched shows are already cached at edge locations.
>
> Netflix 会在高峰时段前用热门内容预热边缘缓存。当晚间观看时段开始时，最
> 迎的节目已预先缓存在边缘节点。

Write-Behind (Write-Back) Caching

For write-heavy workloads, write to cache first and asynchronously persist to dat

针对写入密集型场景，先写入缓存再异步持久化到数据库：

1. Write goes to cache (immediate return to user)

   写入操作直达缓存（立即返回用户响应）

2. Cache acknowledges write 缓存确认写入完成

3. Background process flushes writes to database periodically

   后台进程定期将写入刷新到数据库

This reduces write latency dramatically but introduces risk: if the cache fails bef flushing, writes are lost. Use only when:

这显著降低了写入延迟，但带来了风险：如果在刷新前缓存失效，写入就会丢失。以下情况使用：

- Some data loss is acceptable (analytics counters, view counts)

  允许部分数据丢失（分析计数器、浏览量统计）

- Cache is highly available (Redis with replication and persistence)

  缓存具有高可用性（带复制和持久化功能的 Redis）

- Durability can be sacrificed for performance

  为提升性能可牺牲持久性

You’ve now built a globally distributed system that handles millions of users witl latency worldwide. But the journey doesn’t end here. At truly massive scale, even best off-the-shelf solutions start showing their limits.

现在你已构建出一个全球分布式系统，能为数百万用户提供低延迟的全球服务。但 并未结束。面对真正海量的规模，即便最优秀的现成解决方案也会显露其局限性。

Beyond 10 Million Users

At 10 million users and beyond, you enter territory where off-the-shelf solutions always work. Companies at this scale often build custom infrastructure tailored t

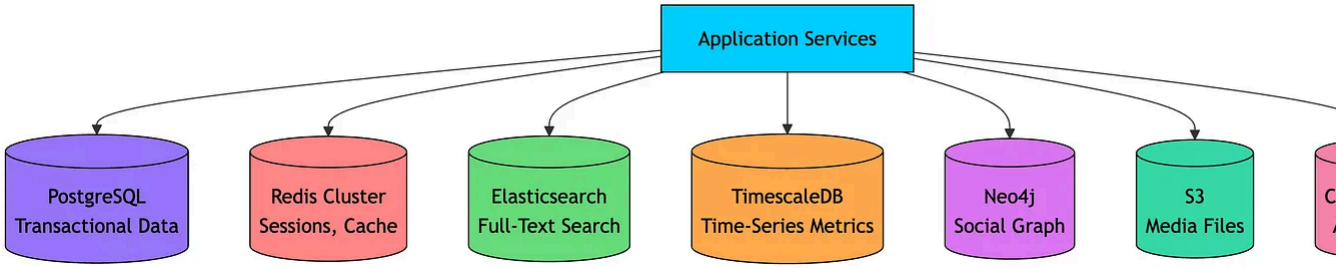
their specific access patterns. The problems become unique to your workload.

当用户规模突破千万量级，你将进入现成解决方案未必适用的领域。达到这种规模
业通常会根据自身特定的访问模式，构建定制化基础设施。此时面临的问题将完全
于你的业务负载特性。

Specialized Data Stores

No single database handles all access patterns well. The concept of “polyglot persistence” means using different databases for different use cases:

没有任何单一数据库能完美处理所有访问模式。"多语言持久化"的概念意味着针对
用例使用不同的数据库：



Data Type	Database Choice	Why This Choice
Transactional	PostgreSQL, MySQL, CockroachDB	ACID guarantees, familiar SQL, mature tooling
Cache/Sessions	Redis, Memcached	Sub-millisecond latency, data structu
Full-text search	Elasticsearch, OpenSearch, Meilisearch	Inverted indexes, relevance scoring, faceted search
Time-series	TimescaleDB, InfluxDB, Prometheus	Optimized for time-ordered writes, downsampling, retention
Graph relationships	Neo4j, Amazon Neptune, Dgraph	Efficient traversal, pattern matching o relationships
Media/Files	S3, GCS, Azure Blob	Cheap, infinitely scalable, durable obj storage
Analytics	ClickHouse, BigQuery, Snowflake	Columnar storage, fast aggregations, massive scale
Key-Value at scale	DynamoDB, Cassandra, ScyllaDB	Predictable latency at any scale, horizontal scaling

Each database is optimized for specific access patterns. Using PostgreSQL for ti series data works but is inefficient. Using Elasticsearch for transactions is possil dangerous.

每个数据库都针对特定的访问模式进行了优化。用 PostgreSQL 处理时间序列数据 可行但效率低下，用 Elasticsearch 处理交易事务则可能带来风险。

Custom Solutions at Scale

At extreme scale, some companies build custom infrastructure because their requirements go beyond what general-purpose systems can deliver:

在超大规模场景下，部分企业会构建定制基础设施，因为通用系统已无法满足其特 求：

- **Facebook's TAO:** A custom data system for the social graph, built to meet Facebook's latency and throughput needs at massive scale when off-the-shel options couldn't.

  Facebook 的 TAO 系统：专为社交图谱设计的定制数据系统，当现成方案无法 脸书在庞大规模下的延迟和吞吐量需求时，该系统应运而生。

- **Google Spanner:** A globally distributed SQL database designed to provide st consistency across regions, combining properties that were hard to get toget the time.

  Google Spanner：一款全球分布式 SQL 数据库，旨在实现跨区域的强一致性 合了当时难以兼得的特性组合。

- **Netflix's EVCache:** A large-scale caching layer built on Memcached, with additional replication, reliability, and operational tooling to support Netflix' traffic patterns.

  Netflix 的 EVCache：基于 Memcached 构建的大规模缓存层，通过额外复制 制、可靠性保障及运维工具支持 Netflix 特有的流量模式。

- **Discord's storage journey:** MongoDB (2015) → Cassandra (2017) → ScyllaDB Each move was driven by the limits of the previous choice, and Discord has s detailed write-ups on the trade-offs behind those migrations.

  Discord 的存储演进史：MongoDB（2015 年）→ Cassandra（2017 年）→ ScyllaDB（2022 年）。每次迁移都源于前代方案的局限性，Discord 已详细拆析这些技术选型背后的权衡考量。

- **Uber's Schemaless:** A MySQL-based storage layer designed to keep transact semantics while scaling beyond a single MySQL setup, with operational sim for teams.

  Uber 的 Schemaless：基于 MySQL 设计的存储层，在突破单机 MySQL 限制时保留事务语义，并为开发团队提供简洁的运维体验。

These aren't options you'll reach for initially, but they illustrate that scaling is an ongoing journey, not a destination. The architecture that works at 1 million user rarely the one you'll want at 100 million.

这些并非初期就会采用的方案，但它们说明扩展是一个持续演进的过程，而非终点用于百万用户的架构，往往难以支撑亿级规模。

Edge Computing

The next frontier is pushing computation closer to users. Instead of all logic run in centralized data centers, edge computing runs code at CDN edge locations worldwide:

下一个前沿领域是将计算推向用户侧。不同于所有逻辑都在集中式数据中心运行，计算让代码在全球 CDN 节点上执行：

- **Cloudflare Workers**: JavaScript/WASM at 250+ edge locations

  Cloudflare Workers：在 250 多个边缘节点运行 JavaScript/WASM

- **AWS Lambda@Edge**: Lambda functions at CloudFront edge

  AWS Lambda@Edge：在 CloudFront 边缘运行的 Lambda 函数

- **Fastly Compute@Edge**: Compute at Fastly's edge network

  Fastly Compute@Edge：Fastly 边缘网络中的计算服务

- **Deno Deploy**: Globally distributed JavaScript runtime

  Deno Deploy：全球分布式 JavaScript 运行时

Edge computing represents a fundamental shift: instead of "request → CDN → o → CDN → response", many requests become "request → edge → response" with t edge having enough compute capability to handle the logic.

边缘计算代表根本性变革：许多请求从传统的"请求→CDN→源站→CDN→响应" 转变为"请求→边缘→响应"模式，边缘节点已具备足够的计算能力来处理业务逻辑

Now that we've covered the full progression from a single server to global-scale infrastructure, an important question remains: how do you know when to take ea step? Scaling too early wastes resources; scaling too late causes outages.

我们已经完整探讨了从单台服务器到全球基础设施的演进过程，但关键问题依然存 如何判断何时该采取每个步骤？过早扩展会浪费资源，过晚扩展则会导致系统崩溃

Summary

Scaling a system from zero to millions of users follows a predictable progression stage solves problems that emerge at specific thresholds:

系统从零用户扩展到数百万用户遵循着可预测的发展路径。每个阶段都针对特定阈 值下出现的问题提供解决方案：

Stage	Users	Key Changes	Main Benefit
1	0-100	Single server, everything together	Ship fast, validate idea
2	100-1K	Separate database from app server	Resource isolation, independe scaling
3	1K-10K	Load balancer, horizontal scaling	Redundancy, handle more tra
4	10K-100K	Caching, read replicas, CDN	Reduce database load, faster reads
5	100K-500K	Auto-scaling, external state	Handle traffic spikes automatically
6	500K-1M	Sharding, microservices, message queues	Scale writes, team independe
7	1M-10M+	Multi-region, CQRS, specialized stores	Global performance, disaster recovery

Key Principles to Remember

1. **Start simple**: Don't optimize for problems you don't have yet. A single serve fine until it isn't.

   从简开始：不要为尚未出现的问题过度优化。单台服务器够用时就无需复杂

2. **Measure first**: Identify the actual bottleneck before adding infrastructure. C bound problems need different solutions than I/O-bound ones.

   先测量后行动：在扩容前准确定位性能瓶颈。CPU 密集型与 I/O 密集型问题 同的解决方案。

3. **Stateless servers are the prerequisite**: You can't horizontally scale or auto-s until your servers hold no local state.

   无状态服务是前提：只有消除服务器本地状态存储，才能实现水平扩展和自 缩。

4. **Cache aggressively**: Most data is read far more often than written. Caching you 10-100x performance improvement for read-heavy workloads.

全力缓存：大多数数据的读取频率远高于写入频率。对于读密集型工作负载，能带来 10-100 倍的性能提升。

5. **Async when possible**: Not everything needs to happen in the request path. E sending, analytics, notifications can all be queued.

   异步优先：并非所有操作都需在请求路径中完成。邮件发送、数据分析、通知等任务均可放入队列处理。

6. **Shard reluctantly**: Database sharding is a one-way door with significant complexity. Exhaust other options first.

   谨慎分片：数据库分片是一条无法回头的复杂之路。务必优先尝试其他优化方

7. **Accept trade-offs**: Perfect consistency and availability don't coexist during network partitions. Know which operations truly need strong consistency.

   接受权衡：在网络分区期间，完美的一致性与可用性不可兼得。明确哪些操作需要强一致性。

8. **Complexity has costs**: Every component you add is a component that can fai needs monitoring, requires expertise to operate.

   复杂性是有代价的：每个新增的组件都可能发生故障，需要监控维护，且依赖运维。

The path to scale isn't about implementing everything at once. It's about understanding which problems emerge at each stage and applying the right solu at the right time.

规模化之路并非要一次性实现所有功能，关键在于识别每个阶段出现的问题，并在时机实施对应解决方案。

The best architecture is the simplest one that meets your current needs, with a cl path to evolve when those needs change.

最佳架构就是满足当前需求的最简方案，同时为未来需求变化预留清晰的演进路径

That's it. Thank you so much for reading!

以上就是全部内容。衷心感谢您的阅读！

If you found this article helpful, give it a like ❤️ and share it with others.

如果觉得这篇文章有帮助，请点赞❤️并分享给他人

For more System Design related content, checkout my website algomaster.io

更多系统设计相关内容，请访问我的网站 algomaster.io

---

119 Likes 119 · 13 Restacks 13

← Previous Next

Discussion about this post

Comments Restacks

Write a comment...

Aryan Dev Shourie · · Jan 30 1 30

Liked by Ashish Pratap Singh · ·

Truly amazing piece! I feel like almost every concept of System Design was covered in this journey. Really appreciate the way you have elaborated each and every system/service for use cases.

/

LIKE (4) REPLY

Potter 1d 1

Liked by Ashish Pratap Singh Ashish Pratap Singh

Thank you very much for sharing your valuable experience with the architecture upgrade

LIKE (1) REPLY

8 more comments... 8 ...

© 2026 Ashish Pratap Singh · Privacy · Terms · Collection notice
Substack is the home for great culture